Define JSON STRUCUTURE

```
{
  "project_name": "E-Commerce Platform",
  "created_date": "April 21, 2024",
  "author": "Project Manager",
  "summary": "Monthly project status and key metrics",
  "key_metrics": [
    {
      "metric": "User Registration",
      "current": "2,450",
      "target": "3,000",
      "status": "On Track"
    },
    {
      "metric": "Revenue",
      "current": "$45,230",
      "target": "$50,000",
      "status": "Behind"
    }
  ],
  "milestones": [
    {
      "name": "Phase 1 Complete",
      "due_date": "April 30, 2024",
      "status": "Completed",
      "notes": "All core features implemented"
    },
    {
      "name": "Beta Launch",
      "due_date": "May 15, 2024",
      "status": "In Progress",
      "notes": "User testing in progress"
    }
  ],
  "risks": [
    {
      "description": "Third-party payment integration delay",
      "impact": "High",
      "mitigation": "Have backup payment processor ready"
    }
  ]
}
```

JavaScript Implementation

Add this code to your `templates.js` file:

```
// ── Project Summary Generator ──────────────────────────────────────────
async function generateProjectSummary(data) {
    const { Document, Packer, Paragraph, TextRun, Table, TableRow, TableCell,
        AlignmentType, BorderStyle, WidthType, ShadingType } = getDocx();

    const C = docxColors();

    // Extract data with defaults
    const projectName = safeStr(data.project_name || 'Project Name');
    const createdDate = safeStr(data.created_date || new
Date().toLocaleDateString());
    const author = safeStr(data.author || 'Author');
    const summary = safeStr(data.summary || 'Project summary');
    const keyMetrics = data.key_metrics || [];
    const milestones = data.milestones || [];
    const risks = data.risks || [];

    // Helper functions
    const bdr = { style: BorderStyle.SINGLE, size: 1, color: C.MGRAY };
    const borders = { top: bdr, bottom: bdr, left: bdr, right: bdr };

    function para(t, bold, color, size) {
        return new Paragraph({
            spacing: { before: 60, after: 60 },
            children: [new TextRun({ text: safeStr(t), bold: !!bold, color: color
|| C.GRAY, size: size || 20, font: 'Arial' })]
        });
    }

    function h1(t) {
        return new Paragraph({
            spacing: { before: 120, after: 120 },
            children: [new TextRun({ text: safeStr(t), bold: true, color: C.NAVY,
size: 32, font: 'Arial' })]
        });
    }

    function h2(t) {
        return new Paragraph({
            spacing: { before: 240, after: 120 },
            children: [new TextRun({ text: safeStr(t), bold: true, color: C.BLUE,
size: 28, font: 'Arial' })]
        });
    }

    function hdrCell(t, w) {
        return new TableCell({
            borders, width: { size: w, type: WidthType.DXA },
            shading: { fill: C.BLUE, type: ShadingType.CLEAR },
            margins: { top: 80, bottom: 80, left: 120, right: 120 },
            children: [new Paragraph({ children: [new TextRun({ text: safeStr(t),
bold: true, color: C.WHITE, size: 18, font: 'Arial' })] })]
```

```
        });
    }

    function dataCell(t, w, bg) {
        return new TableCell({
            borders, width: { size: w, type: WidthType.DXA },
            shading: { fill: bg || C.WHITE, type: ShadingType.CLEAR },
            margins: { top: 70, bottom: 70, left: 120, right: 120 },
            children: [new Paragraph({ children: [new TextRun({ text: safeStr(t),
color: C.GRAY, size: 18, font: 'Arial' })] })]
        });
    }

    function statusCell(t, w) {
        const s = (t || '').toLowerCase();
        let bg, tc;
        if (s.includes('completed') || s.includes('on track')) { bg =
C.LIGHT_GREEN; tc = C.GREEN; }
        else if (s.includes('in progress') || s.includes('pending')) { bg =
C.LIGHT_BLUE; tc = C.BLUE; }
        else if (s.includes('behind') || s.includes('delayed')) { bg =
C.LIGHT_AMBER; tc = C.AMBER; }
        else { bg = C.LIGHT_GRAY; tc = C.GRAY; }
        return new TableCell({
            borders, width: { size: w, type: WidthType.DXA },
            shading: { fill: bg, type: ShadingType.CLEAR },
            margins: { top: 70, bottom: 70, left: 120, right: 120 },
            children: [new Paragraph({ children: [new TextRun({ text: safeStr(t),
bold: true, color: tc, size: 18, font: 'Arial' })] })]
        });
    }

    // Build document sections
    const sections = [];

    // Header section
    const headerChildren = [
        h1(projectName + ' - Project Summary'),
        para('Created: ' + createdDate, false, C.GRAY, 18),
        para('Author: ' + author, false, C.GRAY, 18),
        para(''),
        h2('Executive Summary'),
        para(summary),
        para('')
    ];

    // Key Metrics section
    if (keyMetrics.length > 0) {
        headerChildren.push(h2('Key Metrics'));
        const metricsTable = new Table({
            width: { size: 9360, type: WidthType.DXA },
            columnWidths: [1800, 1400, 1400, 1800, 2960],
            rows: [
                new TableRow({
```

```
                    children: [
                        hdrCell('Metric', 1800), hdrCell('Current', 1400),
                        hdrCell('Target', 1400), hdrCell('Status', 1800),
hdrCell('Notes', 2960)
                    ]
                }),
                ...keyMetrics.map((m, i) => new TableRow({
                    children: [
                        dataCell(m.metric || '', 1800, i % 2 ? C.LIGHT_GRAY :
C.WHITE),
                        dataCell(m.current || '', 1400, i % 2 ? C.LIGHT_GRAY :
C.WHITE),
                        dataCell(m.target || '', 1400, i % 2 ? C.LIGHT_GRAY :
C.WHITE),
                        statusCell(m.status || '', 1800),
                        dataCell('', 2960, i % 2 ? C.LIGHT_GRAY : C.WHITE)
                    ]
                }))
            ]
        });
        headerChildren.push(metricsTable);
        headerChildren.push(para(''));
    }

    // Milestones section
    if (milestones.length > 0) {
        headerChildren.push(h2('Milestones'));
        const milestonesTable = new Table({
            width: { size: 9360, type: WidthType.DXA },
            columnWidths: [2500, 1800, 1800, 3260],
            rows: [
                new TableRow({
                    children: [
                        hdrCell('Milestone', 2500), hdrCell('Due Date', 1800),
                        hdrCell('Status', 1800), hdrCell('Notes', 3260)
                    ]
                }),
                ...milestones.map((m, i) => new TableRow({
                    children: [
                        dataCell(m.name || '', 2500, i % 2 ? C.LIGHT_GRAY :
C.WHITE),
                        dataCell(m.due_date || '', 1800, i % 2 ? C.LIGHT_GRAY :
C.WHITE),
                        statusCell(m.status || '', 1800),
                        dataCell(m.notes || '', 3260, i % 2 ? C.LIGHT_GRAY :
C.WHITE)
                    ]
                }))
            ]
        });
        headerChildren.push(milestonesTable);
        headerChildren.push(para(''));
    }
```

```
    // Risks section
    if (risks.length > 0) {
        headerChildren.push(h2('Risks & Mitigations'));
        const risksTable = new Table({
            width: { size: 9360, type: WidthType.DXA },
            columnWidths: [4000, 1200, 4160],
            rows: [
                new TableRow({
                    children: [
                        hdrCell('Risk Description', 4000), hdrCell('Impact',
1200), hdrCell('Mitigation', 4160)
                    ]
                }),
                ...risks.map((r, i) => new TableRow({
                    children: [
                        dataCell(r.description || '', 4000, i % 2 ? C.LIGHT_GRAY :
C.WHITE),
                        statusCell(r.impact || '', 1200),
                        dataCell(r.mitigation || '', 4160, i % 2 ? C.LIGHT_GRAY :
C.WHITE)
                    ]
                }))
            ]
        });
        headerChildren.push(risksTable);
    }

    sections.push({
        properties: {},
        children: headerChildren
    });

    // Create and pack document
    const doc = new Document({
        sections: sections
    });

    const blob = await Packer.toBlob(doc);
    return {
        blob: blob,
        filename: projectName.replace(/[^a-z0-9]/gi, '_') + '_Summary.docx'
    };
}
```

1. **Added the function** to `templates.js` after the existing `generateTaskBrief` function
2. **Added mode detection** in `loadJSON()` function:

```
 else if (data.project_name) switchMode('projectsummary');
```

3. **Added generation case** in `generate()` function:

```
else if (currentMode === 'projectsummary') result = await
generateProjectSummary(currentData);
```

4. **Added validation** in `showValPanel()` function:

```
else if (currentMode === 'projectsummary') {
   tiles = [
       { l: 'Metrics', n: (data.key_metrics || []).length },
       { l: 'Milestones', n: (data.milestones || []).length },
       { l: 'Risks', n: (data.risks || []).length }
   ];
   if (!data.project_name) { pip = 'warn'; msg = 'No project name found'; }
   else msgs.push({ type: 'ok', text: 'Project summary ready' });
```

5. **Added preview function** in `renderPreview()` function in `templateFunctions.js`:

```
else if (currentMode === 'projectsummary') el.innerHTML =
renderProjectSummaryPreview(data);
```

6. **Add the preview function** (add this after the existing preview functions):

```
function renderProjectSummaryPreview(data) {
    var html = '';
    html += '<div class="doc-cover"><div class="doc-cover-title">PROJECT
SUMMARY REPORT</div>' +
        '<div class="doc-cover-sub">' + escHtml(data.project_name || '') + '
 |  ' + escHtml(data.created_date || '') + '</div>' +
        '<div class="doc-cover-credit">Author: ' + escHtml(data.author || '')
+ '</div></div>';

    if (data.summary) {
        html += '<div class="doc-section-title">Executive Summary</div><div
class="doc-text">' + escHtml(data.summary) + '</div>';
    }

    var metrics = data.key_metrics || [];
    if (metrics.length) {
        html += '<div class="doc-section-title">Key Metrics</div>';
        html += '<table class="doc-table"><thead><tr><th class="th-
blue">Metric</th><th class="th-blue">Current</th><th class="th-
blue">Target</th><th class="th-blue">Status</th></tr></thead><tbody>';
        metrics.forEach(function (m) {
            var status = (m.status || '').toLowerCase();
            var statusClass = status.includes('on track') ||
status.includes('completed') ? 'status-prod' :
                            status.includes('behind') ||
```

```
status.includes('delayed') ? 'status-open' : 'status-other';
            html += '<tr><td><strong>' + escHtml(m.metric || '') + '</strong>
</td><td>' + escHtml(m.current || '') + '</td><td>' + escHtml(m.target || '')
+ '</td><td><span class="status-badge ' + statusClass + '">' +
escHtml(m.status || '') + '</span></td></tr>';
        });
        html += '</tbody></table>';
    }

    var milestones = data.milestones || [];
    if (milestones.length) {
        html += '<div class="doc-section-title">Milestones</div>';
        html += '<table class="doc-table"><thead><tr><th class="th-
navy">Milestone</th><th class="th-navy">Due Date</th><th class="th-
navy">Status</th><th class="th-navy">Notes</th></tr></thead><tbody>';
        milestones.forEach(function (m) {
            var status = (m.status || '').toLowerCase();
            var statusClass = status.includes('completed') ? 'status-prod' :
                              status.includes('in progress') ? 'status-wip' :
'status-other';
            html += '<tr><td><strong>' + escHtml(m.name || '') + '</strong>
</td><td>' + escHtml(m.due_date || '') + '</td><td><span class="status-badge
' + statusClass + '">' + escHtml(m.status || '') + '</span></td><td>' +
escHtml(m.notes || '') + '</td></tr>';
        });
        html += '</tbody></table>';
    }

    var risks = data.risks || [];
    if (risks.length) {
        html += '<div class="doc-section-title">Risks & Mitigations</div>';
        html += '<table class="doc-table"><thead><tr><th class="th-red">Risk
Description</th><th class="th-red">Impact</th><th class="th-
red">Mitigation</th></tr></thead><tbody>';
        risks.forEach(function (r) {
            var impact = (r.impact || '').toLowerCase();
            var impactClass = impact.includes('high') ? 'status-open' :
                              impact.includes('medium') ? 'status-wip' :
'status-other';
            html += '<tr><td>' + escHtml(r.description || '') + '</td><td>
<span class="status-badge ' + impactClass + '">' + escHtml(r.impact || '') +
'</span></td><td>' + escHtml(r.mitigation || '') + '</td></tr>';
        });
        html += '</tbody></table>';
    }

    return html;
}
```

In index.html

inside `gem-links` div add the new buttons.

```
<div id="gem-links" style="display: grid; grid-template-columns: 1fr 1fr; gap:
6px; align-items: stretch;">

   <button class="mode-tab active" data-mode="callscript"
onclick="switchMode('callscript')"
       title="Call Script">📞 Call Script</button>
   <a class="gem-link active" data-modes="callscript"
       href="https://gemini.google.com/gem/18mhrUFqeENrKNekRyVwQMIZ27zUBKFXw?
usp=sharing"
       target="_blank"
       style="display:flex;align-items:center;justify-
content:center;margin:0;text-align:center;font-size:11px;text-
decoration:none;padding:6px;">Call
       Script Gen ↗</a>

   <!-- .... remaining code as it is -->

 </div>
```